

Speech Recognition with SpecAugment

Improving ASR Accuracy via Data Augmentation on Spectrograms
Whisper Fine-Tuning · SpecAugment · WER Evaluation · GUI Application

1. Project Idea

This project is a speech recognition system that converts spoken audio into written text. The system is built using a Whisper speech recognition model and improves its performance by applying data augmentation techniques to the audio features during training.

The main goal of the project is not only to recognize speech, but also to increase the model accuracy compared to normal training. To achieve this, the project uses **SpecAugment**, which modifies spectrograms instead of raw audio. These modifications help the model become more robust against variations in speech speed, missing frequency information, noise, and different speaking styles.

The project also includes a graphical user interface that allows the user to record speech and get the predicted transcription directly.

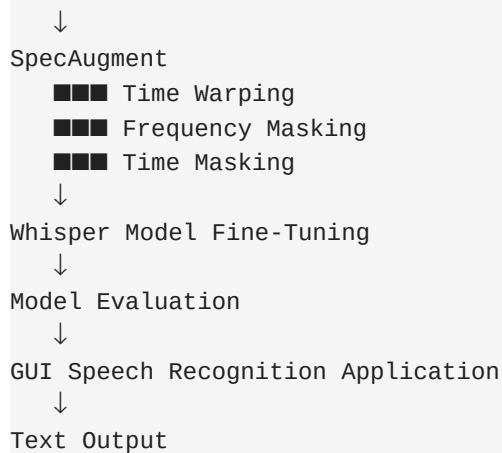
2. Main Objectives

1. Convert audio signals into mel spectrograms.
2. Apply SpecAugment techniques to improve training.
3. Fine-tune a Whisper speech recognition model.
4. Evaluate the model using Word Error Rate and accuracy.
5. Build a simple GUI for real-time speech transcription.
6. Compare normal model performance with the improved fine-tuned model.

3. System Pipeline

The project pipeline follows these stages:

```
Audio Input
↓
Audio Loading and Preprocessing
↓
Mel Spectrogram Conversion
```



4. Dataset Generation

The project uses a small speech dataset made of audio files and their matching text transcripts. Each audio sample has a corresponding sentence that represents the correct transcription.

A data generation script creates speech samples using text-to-speech, then converts them into **.wav** files with a sample rate of 16 kHz. This format is suitable for speech recognition models such as Whisper.

This step is important because the model needs paired data:

Audio file → Correct text

Example:

sample001.wav → "The cat sat on the mat"

5. Audio Preprocessing

Before training or prediction, the audio must be processed into a format that the model can understand. The project loads audio files and extracts:

- The audio signal.
- The sample rate.
- The waveform shape.

Then the audio is converted into a **mel spectrogram**. A mel spectrogram is a visual representation of sound, where one axis represents time and the other represents frequency. This representation is useful because speech patterns become easier for the model to learn.

6. SpecAugment Techniques

The most important added part in this project is **SpecAugment**. Instead of only training the model on clean and fixed spectrograms, the system creates modified versions of the spectrograms. This makes the model stronger and less dependent on exact patterns. The project applies three main

augmentation methods:

7. Time Warping

Time warping changes the position of speech features along the **time axis** of the spectrogram. In normal speech, different people may say the same sentence at different speeds. Some speakers talk slowly, while others talk quickly. Time warping simulates this variation by slightly shifting parts of the spectrogram, helping the model understand speech even if the timing is different from the training examples.

Benefit:

Improves robustness against different speaking speeds.

8. Frequency Masking

Frequency masking hides random **frequency bands** in the spectrogram. In real environments, some frequencies may be unclear because of background noise, microphone quality, or speaker differences. By masking frequency regions during training, the model learns not to depend too much on one specific frequency range.

Benefit:

Improves robustness against noise and missing frequency information.

9. Time Masking

Time masking hides random **time sections** in the spectrogram. This simulates cases where a small part of the speech is unclear, silent, or affected by noise. The model learns to predict the correct text even when some time frames are missing.

Benefit:

Improves robustness when parts of the speech signal are incomplete.

10. Why These Additions Improve Accuracy

A normal model may perform well on clean audio, but its accuracy can decrease when the input speech is different from the training data. Accuracy may decrease if:

- The speaker talks faster or slower.
- There is background noise.
- Some frequencies are weak.
- The microphone quality is different.
- A small part of the audio is unclear.

The added SpecAugment techniques solve this problem by making the training data more variable. The model does not memorize only clean examples — instead, it learns more general speech

patterns. This usually leads to:

- Lower Word Error Rate.
- Higher word accuracy.
- Better generalization.
- Better performance on unseen audio.
- More stable predictions.

11. Model Training

The project fine-tunes a Whisper model using the prepared dataset. The training process includes:

1. Loading the dataset.
2. Splitting it into training and testing data.
3. Loading a pretrained Whisper model.
4. Measuring baseline performance before training.
5. Applying SpecAugment during training.
6. Fine-tuning the model.
7. Saving the improved model.

The model starts from a pretrained Whisper version and is fine-tuned on the project dataset. This is better than training from zero because Whisper already understands general speech patterns.

```
openai/whisper-small.en
```

12. Evaluation

The project evaluates the model using **Word Error Rate (WER)**. WER measures how many word mistakes the model makes compared to the correct sentence. Lower WER means better performance.

Example:

```
Correct:  The cat sat on the mat
Predicted: The cat is on the mat
```

The project also calculates word accuracy:

```
Word Accuracy = 1 - WER
```

The system compares:

```
Before fine-tuning → Baseline WER
After fine-tuning  → Improved WER
```

13. GUI Application

The project includes a graphical user interface for testing the speech recognition system. The GUI allows the user to:

1. Load a pretrained or fine-tuned model.

2. Press a record button.
3. Speak for a fixed number of seconds.
4. Transcribe the recorded speech.
5. Display the predicted text.

The GUI supports two model types:

- A normal pretrained Whisper model.
- A fine-tuned model saved from training.

This makes it easy to compare the normal model and the improved model.

14. Member Contributions

Member	Responsibility	Description
Member 1	Audio Preprocessing	Member 1 handled the audio preprocessing part of the project. They implemented audio loading, normalization, and segmentation.
Member 2	Sparse Image Warping	Member 2 implemented the sparse image warping algorithm. This algorithm is used as the core for time and frequency warping.
Member 3	Time Warping	Member 3 implemented the time warping augmentation technique. This part modifies the spectrogram's time axis.
Member 4	Masking, Training, and Evaluation	Member 4 implemented frequency masking and time masking. They also worked on the training loop and evaluation metrics.
Member 5	Pipeline, Visualization, and Deployment	Member 5 integrated the system into a complete pipeline. They connected preprocessing, augmentation, training, and evaluation into a cohesive workflow.

15. Final System Summary

The final system is a complete speech recognition application with an improved training pipeline. The project begins with audio input, converts it into spectrogram features, applies SpecAugment methods, fine-tunes a Whisper model, evaluates the accuracy, and finally provides a GUI for real-time testing.

The main improvement over a normal speech recognition system is the use of SpecAugment. Time warping, frequency masking, and time masking make the model more robust and help increase accuracy by reducing overfitting and improving generalization.

16. Conclusion

This project demonstrates how speech recognition accuracy can be improved using data augmentation. Instead of training only on normal audio, the system trains on modified spectrograms that simulate real-world speech variations. These additions help the model recognize speech more accurately under different conditions.

By combining Whisper fine-tuning, SpecAugment, evaluation metrics, and a GUI, the project provides both a practical application and a clear method for improving speech recognition performance.